
VORBEREITUNG ZUR VORLESUNG

Handout 02 – Daten in Containern: Bind Mounts und Volumes

Modul	3IT-VSIT-50 · Verteilte Systeme und Internet der Dinge
Thema	Persistenz, Bind Mounts, Volumes
Dozent	Dr. Ulrich Winkler
Semester	5. Semester · Informationstechnik

Das Problem: Container vergessen alles

Ein Container schreibt in seine dünne, beschreibbare Schicht (siehe Handout 01). Diese Schicht lebt und stirbt mit dem Container. Für eine Datenbank oder Logdateien ist das fatal: Beim nächsten Start wären alle Daten weg. Docker bietet deshalb zwei Wege, Daten **außerhalb** dieser Schicht abzulegen.

Bind Mount – ein Host-Ordner im Container

Ein **Bind Mount** blendet einen Ordner Ihres Rechners direkt in den Container ein:

```
docker run -v "$PWD":/app python:3.13-slim ...
```

Alles unter `/app` im Container **ist** Ihr aktueller Ordner. Ändern Sie eine Datei auf dem Host, sieht der Container die Änderung sofort. Das ist ideal für **Code während der Entwicklung**: kein Neubau des Images nötig, um eine Änderung zu testen.

Nachteil: Es hängt vom konkreten Pfad auf dem Wirt ab und ist damit weniger portabel.

Volume – von Docker verwalteter Speicher

Ein **Volume** ist ein Speicherbereich, den Docker selbst anlegt und verwaltet:

```
docker volume create logdaten
docker run -v logdaten:/data python:3.13-slim ...
```

Das Volume überlebt das Löschen des Containers. Ein neuer Container mit demselben Volume findet die Daten wieder. Volumes sind der richtige Ort für **Daten, die bleiben sollen** – Datenbankinhalte, hochgeladene Dateien, unsere Logeinträge.

Gegenüberstellung

	Bind Mount	Volume
Quelle	konkreter Ordner auf dem Wirt	von Docker verwaltet
Typischer Zweck	Code während der Entwicklung	persistente Daten
Sichtbar auf dem Host	ja, direkt	nur über Docker
Portabel	eher nein (Pfadabhängig)	ja
Anlegen nötig	nein	<code>docker volume create</code> (oder implizit)

Faustregel

Code kommt per Bind Mount hinein, Daten leben in einem Volume. In Übung 02 nutzen Sie beides gleichzeitig: den Code-Ordner als Bind Mount, das Datenverzeichnis als Volume.

Und tmpfs?

Es gibt noch **tmpfs-Mounts** – Speicher, der nur im RAM lebt und beim Stoppen verschwindet. Praktisch für flüchtige Geheimnisse oder Zwischenergebnisse, die niemals auf die Platte sollen. In diesem Kurs brauchen wir das nicht, aber es rundet das Bild ab: flüchtig (tmpfs) – an den Host gebunden (Bind Mount) – von Docker verwaltet und dauerhaft (Volume).

Bezug zur Übung

In Übung 02 sehen Sie zuerst, dass ohne Volume jeder `--rm`-Lauf die Logdatei verliert. Dann legen Sie ein Volume an und beweisen, dass die Einträge einen Container-Neustart überstehen. Zum Schluss ändern Sie den Code auf dem Host und sehen die Wirkung sofort – das ist der Bind Mount bei der Arbeit.